witchboard for Cells would be a poor choice. Cells should use DevOps workflows and CI/CD pipelines to kickoff Instrumentor
provisioner jobs in the Amp control-plane, just as early iterations of Dedicated did before Switchboard was developed. It was a design
choice from the start to keep tenant models structured in such a way that an interface would be the same whether one would use
Switchboard or another type of interface.

Amp

Amp is the execution environment in which provisioning jobs are run. This is not required for the execution of provisioning jobs -
in fact, for Dedicated Review apps, Instrumentor runs directly within the GitLab CI pipeline and operates on the developers sandbox
account without the need for Amp. Additionally, Instrumentor is able to be run locally on the environment automation engineers laptop.
This can greatly reduce development time when working on a new feature.

However, the benefit of Amp is that, for Production environments, it is a secure, trusted and highly controlled environment.
Dedicated discourages any long-lived security keys for authentication, instead using trust-relationships, OIDC, identity federation,
and other modern methods for authentication, and Amp provides the control-plane in which this can be done securely.

For security reasons, it is recommended that Cells continue to deploy jobs in the Amp environment. These jobs may however be invoked directly
from CI/CD pipeline jobs via a GitLab Kubernetes Runner transparently. The security, permission and other contexts are then assured at the
Amp level, instead of built within the higher level interfaces.

Tenant Model Schema

Since all input into the provisioning process is done through the tenant model, the model effectively becomes the interface
between the user and/or operator and the provisioner, Instrumentor.

To ensure that all input is consistent and within defined bounds, the tenant model schema validates all input into the system.
This prevents unexpected inputs which could potentially lead to unexpected outcomes or even potentially security breaches or data losses.

Cells: Cells would use its own tenant model schema to define an appropriate schema for Cell instances.

Instrumentor

Instrumentor is the provisioning and configuration tool. The external abstraction into Instrumentor is the tenant model schema, and allows
for different tools to be used, and even replaced over time without affecting the interface or

upstream clients. At present,
the Instrumentor uses Terraform, Ansible, Helm, Kubernetes Manifests.

Cells would either use the Dedicated GCP Instrumentor or use a fork of it. This would require further analysis before a decision is made.

Tenant Control: tenctl

tenctl, or tenant controller, is a golang binary used extensively during the provisioning process. It also supports execution locally,
for development-time usage, and in GitLab CI/CD.

tenctl helps bridge the gaps between the various tools (such as Terraform, Ansible) and tenant data. This includes:

1. Handling configuration of underlying tools, such as Cloud Provider configuration files (.awscredentials.ini, AWSREGION environment variables, etc)
2. Handling authentication and authorization
3. A single method for templating the configuration files required by GET, Ansible, Terraform, etc (using Jsonnet, which can generate YAML, ini, JSON and all other required command types)
    1. In similar projects, this templating is normally done using a wide variety of tools (such as jq, Go templates, envsubst, string concatenation in bash scripts etc)
    2. Having a single tool allows much more consistent templating, plus testing harnesses, etc
4. Tenant model schema validation using JSON schema.
5. Tenant model migration tools to ensure that tenant models can be upgraded automatically without manual intervention
6. Cloud provider orchestration not covered by Ansible, Terraform, etc, for example for checking the status of backups.

In many provisioning tools, the functionality that tenctl provides is typically written as a set of bash scripts. These scripts are notoriously unstructured,
unscalable, difficult to test and verify. This leads to slow-downs in development and even bugs or data loss during provisioning or day-two operations.

Writing this tool in Go provides benefits of a single tool which operates identically in all environments (no differences between shelling out to command line
tools in MacOS and different versions of linux) and is fully tested. Additionally, Go encourages better structure and ensures the structure can scale
as the complexity of the project increases over time.

The alternative to tenctl is something we see too often on many Infrastructure projects today: dozens of unmaintained · and unmaintainable ·
untested bash scripts, each with its own input, arguments, subset of environments in which it can run, each with it's own way of converting input
data into configuration data. This leads to complexity, inconsistency, cognitive load and ultimately lower velocity and even outages due to unexpected
behaviours (aka: "it worked on my machine").

To illustrate this with an analogy: it's possible to interact with a Kubernetes cluster using only bash and curl. However, using kubectl allows for

a much more consistent and scalable client. tenctl is similar to kubectl, but focused on GitLab Tenant instances instead of Kubernetes instances.

Cells benefit greatly from using a tool like tenctl. By ensuring that there is a single way to template configuration across different tools,
we avoid the drift that exists currently in GitLab.com environments.

How Dedicated Could Be Used to Support Cells

Design Considerations

Many of the same design considerations that apply for GitLab Dedicated apply equally well to the deployment of Cells.

1. A large number of homogeneous environments will be required
2. Each potential difference between environments will result in the number of combinations of those differences growing exponentially
3. Defining the environments as multiple independent modules would quickly become unsustainable to maintain
4. The larger the number of potential combinations:
    1. The more difficult environmental automation of day-two operations becomes
    2. The less testable a change becomes
5. Having homogeneous GitLab environments with a single atomic provisioner allows changes to be rolled out gradually across a large number of environments. This is not dissimilar to how a replica set gradually rolls out new pods in Kubernetes, replacing older versions safely, until all old pods have been replaced.

Deploying Cells Using Environment Automation

Cells could use the same deployment model as Environment Automation used before Switchboard had been developed. This is represented by the switchboardla project.

Deploying Cells using Environment Automation

In this deployment model, we forgo a graphical user interface, and provide a devops model. This approach is much more in keeping with the way SREs work.
A project is maintained on a GitLab instance, and the tenant models are kept as files in the git repository.

All change is controlled via merge requests with the standard GitLab approval approaches.

When changes are merged to the main branch, they kick off CI pipelines, which in turn kick off jobs in the Amp environment. SREs can monitor the progress
of changes using standard GitLab tooling, such as CI traces, etc.

SECTION: engineering/infrastructure/team/runway/_index.md

Mission

The Runway team builds and supports the Runway product, which aims to enable teams to deploy their code to staging & production quickly, safely and efficiently.

Ownership

The team has ownership over the following areas:

Services

The Runway team handles issues labeled Service::Runway

For services deployed to Runway, the Runway team is responsible for building functionality for the platform, operating the platform, and maintaining the platform for service owners.

Production Readiness Review

The Runway team is responsible for Production Readiness Review process in collaboration with various subject matter experts. The purpose of the Runway team owning PRR is to ultimately build production-readiness criteria directly into the platform.

CustomersDot

The Runway team is responsible for CustomersDot infrastructure support. The purpose of the Runway team owning operational support is to ultimately migrate CustomersDot into the platform (epic).

Getting Assistance

Should you require assistance from the Runway team, please reference getting assistance in Infrastructure.

Team Members

{{< team-by-manager-slug "jtoto-gtl" >}}

Common Links

- Runway Top Level Epic
- grunway - work and team related discussions.
- frunway - external requests, questions, support.

How We Work

Project Management

The Runway team has a single top level Epic that links sub-epics for all projects the team is working on.
Every sub-epic has a completion date and a DRI for providing a weekly status update.

We follow Platforms Project Management practices as outlined in the Handbook. You can read our 2024 Team Impact Review

Issues

Issues for work performed by the Runway team is located in the Runway team issue tracker. The exception to this is work that is part of the Rails Application, these issues should be opened in the GitLab Rails issue tracker.

SECTION: engineering/infrastructure/team/scalability/_index.md

Common Links

Workflow	Team workflow
GitLab.com	@gitlab-org/scalability
Issue Trackers	Scalability
Team Slack Channels	gscalability - Company facing channel gscalability-observability - Team channel gscalability-practices - Team channel scalabilitysocial - Group social channel
Information Slack Channels	infrastructure-lounge (Infrastructure Group Channel), incidents (Incident Management), alerts-general (SLO alerting), mechsympalerts (Mechanical Sympathy Alerts)

Project Management Links

Group Level

1. Scalability Epic Board
1. Scalability Issue Board
1. Scalability Issues not in an Epic
1. Scalability Issues by Team
1. Scalability Issues by Team Member

Mission

The Scalability group is responsible for GitLab at scale, working on the highest priority scaling items related to our SaaS platforms.
We support other Engineering teams by sharing data and techniques so they can become better at scalability as well.

Vision

As its name implies, the Scalability group enhances the availability, reliability and, performance of GitLab's SaaS platforms
by observing the application's capabilities to operate at scale.

The Scalability group analyzes application performance on GitLab's SaaS platforms, recognizes bottlenecks in service availability, proposes (and develops) short term improvements and develops long term plans that help drive the decisions of other Engineering teams.

Short term goals for the group include:

Refine existing, define new, and document Service Level Objectives for each of GitLab's services.
Continuously expose the top 3 critical bottlenecks that threaten the stability of our SaaS platforms.
Work on scoping, planning and defining the implementation steps of the top critical bottleneck.
Define and track team KPI's to track impact on our SaaS platforms.
Work on implementing user facing application features (such as API improvements) as a means to reduce pressure on our SaaS platforms generated by regular user interactions.

Direction for FY24

We've moved the direction to the direction section here so that it's in the same place as the rest of our product direction.

Indicators

The Infrastructure Department is concerned with the availability and performance of GitLab's SaaS platforms.

GitLab.com's service level availability is visible on the SLA Dashboard, and we use the General GitLab Dashboard in Grafana to observe the service level indicators (SLIs) of apdex, error ratios, requests per second, and saturation of the services.

These dashboards show lagging indicators for how the services have responded to the demand generated by the application.

Each team is responsible for separate indicators. For more information, please view the team pages linked above.

Themes

The broad nature of work undertaken by the Scalability group can make prioritization challenging as it's tricky to compare some issues like-for-like. For example, how do we compare the benefit of an issue to address a performance concern against an issue that reduces developer toil? To help guide the direction of the group and to inform our prioritization process, we can categorize issues in to the following themes, in order of priority:

1. Critical Saturation Response. On occasions saturation alerts can unexpectedly occur - for example, when caused by a sudden change in platform usage patterns - and need to be addressed with urgency. We try to avoid working reactively by proactively working on other themes.
1. Horizontal Scalability. The most obvious scaling bottlenecks in our infrastructure are those that can only be scaled vertically instead of horizontally. Horizontal scaling brings the

benefit of elasticity, which increases confidence that we can meet future demand while keeping costs linear - both of these elements are strongly aligned with the vision of the Scalability group.
1. Increasing Plaform Capacity. Delivering foundational project work in the GitLab application and infrastructure to support service capacity needs for GitLab SaaS.
1. Scalability Advocacy and Facilitation. An effective method for the Scalability group to leverage its output is by collaborating closely with other engineering teams to promote scalability best practises. This might include building tools to enable wider engagement in GitLab SaaS operations (e.g. Stage Dashboards), or serving as a point of contact to other teams for scaling questions relating to their own initiatives.
1. Eliminating Toil. We want to make our output as efficient as possible by spending more time on engineering projects and less time on manual, repetitive, or automatable tasks. An effective way of achieving this is by considering how future toil can be avoid when delivering projects. However, inline with our Iteration value, we don't want to over-optimize and we can't consider all eventualities ahead of time. We should always be mindful of opportunities to reduce toil, which will make us more effective in the long-term.

The above list is not comprehensive, nor does it outline a formal process. We should remain pragmatic when prioritizing work, while using the themes as a guideline.

Job Families

The Scalability Group consists of a Senior Engineering Manager, Engineering Managers, Backend Engineers, and Site Reliability Engineers.

The Engineering Roles section of the handbook lists the responsbilies of these roles:

Engineering Manager
Backend Engineer with Scalability specialization.
Site Reliability Engineer with Scalability specialization.

Working with us

Emergency Escalation during S1/S2 incidents

Scalability leadership can be reached via PagerDuty Scalability Escalation.

From https://gitlab.pagerduty.com/incidents, click on the "New Incident" button and complete the new incident form as shown below.

How do I engage with the Scalability Group?

1. Start with an issue in the Scalability tracker: Create an issue.
1. You are welcome to follow this up with a Slack message in gscalability.
1. Please don't add any workflow labels to the issue. The team will triage the issue and apply these.

Alternatively, mention us in the issue where you'd like our input.

When issues are sent out way, we will do our best to help or find a suitable owner to move the issue forward.
We may be a development team's first contact into the Infrastructure department and we endeavour to treat these
requests with care so that we can help to find an effective resolution for the issue.

Scalability review requests

If you're working on a feature that has specific scaling requirements, you
can create an issue with the review request template.
Some examples are:

1. Review Request - Impact on database load for enabling advanced global search
1. Review Request - Assumptions about build prerequisite-related application limits
1. Review Request - Throttling for Cleanup Policies Scaling Request

This template gives the Scalability group the information we need to help you, and the issue will be shown on
our build board with a high priority.

How does the Scalability Group engage with Stage Groups?

When we observe a situation on GitLab.com that needs to be addressed alongside a stage group, we first raise an issue
in the Scalability issue tracker that describes what we are seeing. We try to determine if the problem lies with the action
the code is performing, or the way in which it is running on GitLab.com. For example, with queues and workers, we will see
if the problem is in what the queue does, or how the worker should run.

If we find that the problem is in what the code is doing, then we engage with the EM/PM of that group to find the right path
forward. If work is required from that group, we will create a new issue in the gitlab-org
project and use the Availability and Performance Refinement process to highlight this issue.

How we work

Handbook First

In line with the broader GitLab culture, we adopt a Handbook First approach to documenting our team's workflow. Should you have any proposals aimed at enhancing our processes, please initiate a Merge Request (MR) to update the handbook. Assign the MR to @rnienaber for the Group level change and Scalability EMs for the respective team changes and tag the team in a comment to solicit feedback. If there are no objections within three working days of tagging the team,
the MR will be deemed ready for merging. We adhere to the principle of making two-way door decisions meaning additional MRs can be created to suggest changes or removals of processes that are deemed inefficient.

Communication

Everything is written in epics, issues, runbooks or the handbook.
Decisions or important information in Slack must be copied into a relevant location so that the information is persisted beyond the 90-day Slack retention policy.

Slack Channels and Guidelines

We communicate in public using the following channels:

1. gscalability - Company facing channel where other team members can reach out to us. We also use this channel for highlighting work we have done.
1. gscalability-observability - Team channel where daily work information is shared and team coordination takes place.
1. gscalability-practices - Team channel where daily work information is shared and team coordination takes place.
1. scalability-social - Group social channel.

In the team channels, team members are encouraged to share what they are working and any blockers they may have.
The format is not fixed so that people share in a way that feels natural to them.

In addition to the channels above, we create a project channel per epic when appropriate.
A channel dedicated to a project helps to keep everything about the topic in one place and makes it easy to stay up to date on that topic.
It is useful for teams working across time zones or for getting back up to speed after being on leave.
The DRI for a project owns the project Slack channel.

Meetings and Scheduled Calls

We prefer to work asynchronously as far as possible but still use synchronous communication where it makes sense to do so.
Asynchronous communication is the best way to make sure everyone, regardless of timezone or availability, is included.

To keep people connected, team members are encouraged to schedule at least one coffee-chat with another team member each week.

Demo Calls

There is one demo call per week.
The time of the call differs each week to try to get people from different timezones to join different calls.
The purpose of this call is to showcase something that you have been working on during that week. It does not have to be perfect or polished.
These calls are a technical conversation and while we might land up with guidance on what we are working on, the purpose is not to make fixed decisions on this call.

Items should be added to the agenda ahead of the meeting.

If there are no agenda items at that time, the call is cancelled for that week.

The call should be recorded and added to GitLab Unfiltered. Please use your discretion when choosing the visibility level as some screen shares contain private data. If you upload a private video, please add information in the description for why this visibility was chosen.
A playlist of the recordings is available on GitLab Unfiltered.

Communicating our work schedule to others

It is important that team-members know when we are available, so we keep our calendars updated.

1. We use the "working hours" settings in Google calendar.
1. We indicate "async only" periods if we need them during our working hours.
1. Our PTO tracker is linked to the group Google calendar using gitlab.com3puidsh74uhqdv9rkp3fj56af4@group.calendar.google.com as an additional calendar in the Calendar Sync settings.
1. Any team members with on-call responsibilities should share their Pager Duty calendar with their manager.

Impact

As a small team covering a wide domain, we need to make sure that everything we do has sufficient impact. If we do something that only the rest of the Scalability group knows about, we haven't 'shipped' anything. Our 'users' in this context are the infrastructure itself, SREs, and Development Engineers.

Impact could take the form of changes like:

1. Development practices that make it easier for our Development department to ship code that works well at scale.
2. Monitoring changes that mean we can detect and attribute problems sooner, particularly focusing on utilisation.
3. Code changes that reduce pressure on a part of our system that's feeling the strain.
4. Guides for Developers and SREs to work with a given service.

Announcing Impactful Items

In order to make others aware of the work we have done, we should advertise changes in the following locations:

1. Engineering Week-in-Review
1. Slack Channels
   1. For Backend Engineers
      1. development
      1. eng-managers
      1. devtipoftheday
      1. development-guidelines
   1. For SRE's

1. infrastructure-lounge
1. infra-staff

When collaborating on the announcement text, consider using a threaded discussion on the relevant epic, issue, or change request.

Documentation or tutorial videos should also be added to the README.md in our team repository.

Triage rotation

We have automated triage policies defined in the triage-ops project. These perform tasks such as automatically labelling issues, asking the author to add labels, and creating weekly triage issues.

We rotate the triage ownership each month, with the current triage owner responsible for picking the next one (a reminder is added to their last triage issue).

Triaging issues

When issues arrive on our backlog, we should consider how they align with our vision, mission, and current OKRs.

We also determine which of the teams would be the more appropriate owner for that task.

We need to effectively triage these issues so that they can be handled appropriately. This means:

1. Critically assess the issue to understand the problem
1. Determine if this impacts .com or Self-Managed instances.
   1. If this primarily affects Self-Managed instances, the issue can usually be redirected to the Application Performance group.
1. If this is a scaling issue, assign it into our backlog using workflow labels and place it on the planning board if necessary.
1. If this is not a scaling issue, find the most appropriate owner in either Infrastructure or Development, or any other department.

When handing over an issue to the new owner, provide as much information as you can from your assessment of the issue.

Engagement with Incidents

The Scalability team members often have specialized knowledge that is helpful in resolving incidents. Some team members are also SREs who are part of the on-call rota. We follow the guidelines below when contributing to incidents.

For an on-call SRE:

follow the ordinary incident management procedures for EOC

at the end of your shift, if there are active incidents or corrective actions in progress, inform the EM who will help you to prioritize the remaining work

For an Incident Manager:

follow the ordinary incident management procedures for IM
handover should occur as normal, but inform the EM if there are active incidents you are still working on

If you are not EOC or an Incident Manager when an incident occurs:

For S1 incidents
  the priority is to get GitLab.com up and running and getting back to a stable state takes priority over project work
  when the system is stable, contribute to determining the root cause and writing up the corrective actions
  the IM or Reliability EM will delegate corrective actions
  work with the Scalability EM to prioritize any work that arises from an S1
For all other incidents
  if you are called into an incident, the priority is to enable others to resolve the problem
  the expectation is to be hands-off, giving guidance where necessary, and returning to project work as soon as possible

The reason for this position is that our project work prevents future large S1 incidents from occurring.
If we try to participate in and resolve many incidents, our project work is delayed and the risk of future S1 incidents increases.

Engagement with the Infradev Process

The Infradev process aims to highlight SaaS
availability and reliability improvements with the Stage Groups.

Where issues marked as infradev are found to be scaling problems, the team::Scalability label should be added.

Our commitment to this process, in line with the team's vision, is to provide guidance and assistance to the stage groups who are responsible for resolving
these issues. We proactively assist them to determine how to resolve a problem, and then we contribute to reviewing
the changes that they make.

Weekly Issues

1. Service::Unknown refinement - go through issues marked Service::Unknown and add a defined service, where possible.
1. Review request processing - the goal is to move review request issues to workflow-infra::In Progress, either
  through picking them up directly, or asking on our team channel if any one else is able.

Monthly Issues

1. Infradev review - show issues with team::Scalability and infradev labels so we can help the stage groups move those forward.

Quarterly Issues

Every quarter, we perform a review of all issues on the backlog that are not part of any project. When reviewing issues:

if the issue is no longer relevant then it should be closed
if it is relevant but we are unlikely to work on it soon it should remain open

The EM creates this issue each quarter. It is not the sole responsibility of the person on Triage Rotation and is shared among all team members.

Regarding Coding at Scale

Software development often happens on a single machine, with a single application version and almost no load.

This configuration is very different from what happens on GitLab.com and our customers' installations.

The problem "at scale" comes from a different order of magnitude than the development and testing environments.
Things like the number of servers, the number of incoming requests, the number of rows on a table or
the number of application versions will make the difference between something that works on your computer and
something that works in production.

An extra challenge, almost unique to GitLab, is that we deploy from the main branch multiple times each day, but we have a monthly release cycle
and zero downtime updates is a requirement for both releases.

Overlooking the compatibility with multiple versions of the application running at the same time
can induce a production incident.

You can find more detailed information in the links below. If this is not enough, please reach out to the
delivery or
scalability team.

1. Expand and Contract pattern
2. Zero Downtime Updates
3. Sidekiq Compatibility across Updates
4. Avoiding downtime in migrations
5. Uploads development documentation

Team History

The Scalability team became a reality during the fourth organizational iteration in the Infrastructure department on 2019-08-22, although it only became a reality once the first team member joined the team on 2019-11-29.

Even though it might not look like it at first glance, the Scalability team has its origin connected to the Delivery team. Namely, the first two backend engineers with Infrastructure specialisation were a part of the Delivery team, a specialisation that previously did not fit into the organizational structure. They had a focus on reliability improvements for GitLab.com, often working on features that had many scaling considerations. A milestone, that will prove to be a case for the Scalability team, was Continuous Delivery on GitLab.com.

Throughout July, August and September 2019, GitLab.com experienced a higher than normal amount of customer facing events. Mirroring delays, slowdowns, vertical node scaling issues (to name a few) all contributed to general need to improve stability. This placed higher expectations on the Infrastructure department and with the organization at the time, this was harder to meet. To accelerate the timelines, "infradev" and "rapid action" processes were created, as a connection point between Infrastructure and Development departments to help Product prioritise higher impact issues. This approach was starting to yield results, but the process was there as a reaction to an (ongoing) event with the focus on resolving that specific need.

The background processing architectural proposal clearly illustrated the need to stay ahead of the growing needs of the platform and approach the growth strategically as well as tactically. With a clear case and approvals in hand, the team mission, vision, and goals were set and the team buildout could commence. While that was in motion, we had another confirmation through a performance retrospective that the need for the team is real.

As the team was taking shape, the background processing architectural changes were the first changes delivered by the team with a large impact on GitLab.com, with many more incremental changes throughout 2020 that followed. Measuring that impact reliably, and predicting the future challenges remains one of the team focuses at the time of the writing of this history summary.

The team impact overview is logged in issues:

1. Year overview for 2020
1. Year overview for 2021
1. Year overview for 2022
1. Year overview for 2023

SECTION: engineering/infrastructure-platforms/_index.md

Mission

As Infrastructure Platforms, our mission is to enable GitLab to deliver a single DevSecOps platform across SaaS and self-managed platforms by building highly available, reliable, performant, and scalable infrastructure solutions while maintaining the lowest total cost of

ownership.

Vision

Deliver the industry leading SaaS solutions, empowering organizations worldwide with the most innovative and efficient DevSecOps platform.

Getting Assistance

If you're a GitLab team member and are looking to alert the Infrastructure Platforms teams about an availability issue with GitLab.com, please find quick instructions to report an incident here: Reporting an Incident.

For all other queries, please see the getting assistance page.

Direction

Initiatives driven within the Platforms section, often spanning multiple quarters, are represented on the SaaS Platforms section epic (GitLab team member).

{{% include "includes/we-are-also-product-development.md" %}}

Organization structure

(click the boxes for more details)

```
mermaid
flowchart LR
    I[Infrastructure Platforms]
    click I "/handbook/engineering/infrastructure-platforms/"

    I --> DA[Data Access]
    click DA "/handbook/engineering/infrastructure-platforms/data-access/"
    I --> DE[Developer Experience]
    click DE "/handbook/engineering/infrastructure-platforms/developer-experience/"

    I --> GD[GitLab Dedicated]
    click GD "/handbook/engineering/infrastructure/team/gitlab-dedicated/"
    I --> PRODENG[Production Engineering]
    click PRODEND "/handbook/engineering/infrastructure/team/production-engineering/"
    I --> SD[GitLab Delivery]
    click SD "/handbook/engineering/infrastructure-platforms/gitlab-delivery/"
    I --> TS[Tenant Scale]
    click TS "/handbook/engineering/infrastructure-platforms/tenant-scale/"

    DA --> DF[Database Framework]
    click DF "/handbook/engineering/infrastructure-platforms/data-access/database-framework/"
    DA --> DO[Database Operations]
    click DO "/handbook/engineering/infrastructure-platforms/data-access/database-operations/"
    DA --> Durability
```

```
    click Durability "/handbook/engineering/infrastructure-platforms/data-access/durability/"
    DA --> Git
    click Git "/handbook/engineering/infrastructure-platforms/data-access/git/"
    DA --> Gitaly
    click Gitaly "/handbook/engineering/infrastructure-platforms/data-access/gitaly/"

    PRODENG --> Ops
    click Ops "/handbook/engineering/infrastructure/team/ops/"
    PRODENG --> Foundations
    click Foundations "/handbook/engineering/infrastructure-platforms/production-
engineering/foundations/"
    PRODENG --> Runners[Runners Platform]
    click Runners "/handbook/engineering/infrastructure-platforms/production-
engineering/runners-platform/"
    PRODENG --> Observability
    click Observability "/handbook/engineering/infrastructure-platforms/production-
engineering/observability/"
    PRODENG --> Runway
    click Runway "/handbook/engineering/infrastructure/team/runway/"
    PRODENG --> CC[Cloud Connector]
    click CC "/handbook/engineering/infrastructure/team/cloud-connector/"
    PRODENG --> FO[FinOps]
    click FO "/handbook/engineering/infrastructure/team/finops/"

    DE --> DevA[Development Analytics]
    click DevA "/handbook/engineering/infrastructure-platforms/developer-
experience/development-analytics/"

    DE --> DT[Developer Tooling]
    click DT "/handbook/engineering/infrastructure-platforms/developer-experience/developer-
tooling/"
    DE --> FR[Feature Readiness]
    click FR "/handbook/engineering/infrastructure-platforms/developer-experience/feature-
readiness/"
    DE --> PER[Performance Enablement]
    click PER "/handbook/engineering/infrastructure-platforms/developer-experience/performance-
enablement/"
    DE --> TG[Test Governance]
    click TG "/handbook/engineering/infrastructure-platforms/developer-experience/test-
governance/"

    GD --> E[Environment Automation]
    click E "/handbook/engineering/infrastructure/team/gitlab-dedicated/environment-
automation/"
    GD --> PSS[Public Sector Services]
    click PSS "/handbook/engineering/infrastructure/team/gitlab-dedicated/us-public-sector-
services/"
    GD --> Switchboard
    click Switchboard "/handbook/engineering/infrastructure/team/gitlab-dedicated/switchboard/"
```

```
    SD --> B[Build]
    click B "/handbook/engineering/infrastructure-platforms/gitlab-delivery/distribution/"
    SD --> Framework
    click Framework "/handbook/engineering/infrastructure-platforms/gitlab-delivery/framework/"
    SD --> R[Release]
    click R "/handbook/engineering/infrastructure-platforms/gitlab-delivery/delivery/"
    SD --> D[Deploy]
    click D "/handbook/engineering/infrastructure-platforms/gitlab-delivery/delivery/"

    TS --> Organizations
    click Organizations "/handbook/engineering/infrastructure-platforms/tenant-
scale/organizations/"
    TS --> CI[Cells Infrastructure]
    click CI "/handbook/engineering/infrastructure-platforms/tenant-scale/cells-
infrastructure/"
    TS --> Geo
    click Geo "/handbook/engineering/infrastructure-platforms/tenant-scale/geo/"
```

Dogfooding

The Infrastructure Platforms department uses GitLab and GitLab features extensively as the main tool for operating many environments, including GitLab.com.

We follow the same dogfooding process as part of the Engineering function, while keeping the department mission statement as the primary prioritization driver. The prioritization process is aligned to the Engineering function level prioritization process which defines where the priority of dogfooding lies with regards to other technical decisions the Infrastructure Platforms department makes.

When we consider building tools to help us operate GitLab.com, we follow the 5x rule to determine whether to build the tool as a feature in GitLab or outside of GitLab. To track Infrastructure's contributions back into the GitLab product, we tag those issues with the appropriate Dogfooding label.

Handbook use at the Infrastructure Platforms department

At GitLab, we have a handbook first policy. It is how we communicate process changes, and how we build up a single source of truth for work that is being delivered every day.

The handbook usage page guide lists a number of general tips. Highlighting the ones that can be encountered most frequently in the Infrastructure Platforms department:

1. The wider community can benefit from training materials, architectural diagrams, technical documentation, and how-to documentation. A good place for this detailed information is in the related project documentation. A handbook page can contain a high level overview, and link to more in-depth information placed in the project documentation.
1. Think about the audience consuming the material in the handbook. A detailed run through of a GitLab.com operational runbook in the handbook might provide information that is not applicable to self-managed users, potentially causing confusion. Additionally, the handbook is not a go-to

place for operational information, and grouping operational information together in a single place while explaining the general context with links as a reference will increase visibility.
1. Ensure that the handbook pages are easy to consume. Checklists, onboarding, repeatable tasks should be either automated or created in a form of template that can be linked from the handbook.
1. The handbook is the process. The handbook describes our principles, and our epics and issues are our principles put into practice.

Projects

Classification of the Infrastructure Platforms department projects is described on the infrastructure department projects page.

The infrastructure issue tracker is the backlog and a catch-all project for the infrastructure teams and tracks the work our teams are doing·unrelated to an ongoing change or incident.

In addition to tracking the backlog, Infrastructure Platforms department projects are captured in our Infrastructure Platforms department Epic as well as in our Quarterly Objectives & Key Results

Supporting Product Features

We have a model that we use to help us support product features. This model provides details on how we collaborate to ship new features to Production.

How we work

Communication

Slack

Our main method of communication is Slack.

If you need assistance with a production issue or incident, please see the section on getting assistance.

SaaS Platforms

Channel	Purpose
infrastructure-platforms	We collaborate on department level items here. This channel is used to share important information with the wider team, but also serves to align all teams in Platfroms with the common topic.
ginfrastructureplatformsleads	Communication for managers. Everyone interested is welcome to join this channel if they find the topics interesting.
confidential managers channel	Used to discuss staffing issues affecting all teams that require additional coordination. We default to using the public channel as much as possible.
infrastructureplatformssocial	Our social channel.

Dedicated

Channel	Purpose
gdedicated-team	Dedicated Group discussion channel. Please use this channel for discussions relevant to engineers across the Dedicated group
fgitlabdedicated	Dedicated function channel. Please use this channel to ask questions about features or ways of using the Dedicated product. Dedicated group will use this channel to make announcements relevant to wider groups
gdedicated-us-pubsec	Dedicated USPubSec team channel. Used to discuss topics that affect PubSec team only. For broader engineering discussions please use gdedicated-team
gdedicated-switchboard-team	Dedicated Switchboard team channel. Used to discuss topics that affect Switchboard team only. For broader engineering discussions please use gdedicated-team
gdedicated-environment-automation-team	Dedicated Environment Automation team channel. Used to discuss topics that affect Switchboard team only. For broader engineering discussions please use gdedicated-team
gdedicated-team-social	Dedicated social channel
dedicated-mr-review-stream	Visibility of new merge requests on Dedicated repos

Delivery

Channel	Purpose
gdelivery	Delivery Group channel
gdeliverystandups	
deliverysocial	Social channel for the group.
releases	General communication about the current Release/Patch
fupcomingrelease	Detailed Release status / Release Manager channel
announcements	Release-Tools automation posts related to deployment activity

Production Engineering

Channel	Purpose
sproductionengineering	General conversation for Production Engineering teams and requests coming in from other team members.
gproductionengineeringleads	Channel for Production Engineering leads (staff+ and management)
ginfraops	Team channel for Production Engineering Ops
ginfrasocial	Social channel for Production Engineering
gfoundations	Team channel for Production Engineering Foundations
gfoundationssocial	Social channel for the Foundations team
gfoundationsalerts	Non-urgent service alerts for Foundations owned services
gfoundationsnotifications	Renovate notifications for Foundations owned projects
infra-terraform-alerts	Terraform state drift alerts for SaaS infrastructure
grunnersplatform	Team channel for Production Engineering Runners Platform
gobservability	Team channel for general work in Observability.
grunway	Team channel for general work and discussion.
rrunway	External channel used for support, requestions, and help with Runway.

Tenant Scale

Channel	Purpose
stenantscale	General conversation for Tenant Scale and requests coming from other teams.
gorganizations	Discussions and requests specific to the Organizations team.
ggeo	Discussions and requests specific to the Geo team.
gcellsinfrastructure	Discussions and requests specific to the Cells Infrastructure team.
fcellsandorganizations	Channel for cross-functional discussion and coordination on Cells and Organizations.

The SaaS Platforms group is gradually directing requests for help to the saas-platforms-help Slack channel.
This channel can be used if it is unclear which Infrastructure team the question should be directed to.
For more information, refer to the landing page for getting assistance.

The saas-platforms-help channel is monitored by SaaS Platforms Engineering Managers and Staff+ engineers who triage any inbound requests. When triaging this channel, one should locate the team who can best answer this question and instruct the requestor to contact that team using the team's preferred contact method. When the requestor is connected to the right team, add a green check emoji to the message. Finally, if needed, update the getting assistance page with any changes.

Meetings

Once per week, we hold a Platforms leads call to align on action items related to career development, general direction or answer any ongoing questions that have not been addressed async. The call is cancelled when there are no topics added on the morning of the call.

In addition to the Platforms leads call, we have some recurring events and reminders that can be viewed in the SaaS Platforms Leadership Calendar. Please add this to your Calendars to stay up-to-date with the various events.

Sr. Director of Infrastructure Marin Jankovski, likes to meet with new team members that join the organization. Marin sets up informal 1:1 coffee chats a few times a month with newer team members to get to know one another and see how they are doing. This process is organized by his EBA who will reach out to team members once he has the availability to meet. As this is a large team, it may take a while to get through everyone.
If someone needs to meet with Marin sooner than when the coffee chat is scheduled, you can reach out to his EBA Liki Simonot to set something up.

Grand Review

The Engineering Leads for each Stage, along with their Product Managers, hold weekly progress reviews to assess their groups· progress, share project updates, resolve blockers, and celebrate wins. Additionally, the Director of Product and the Senior Director of Infrastructure Platforms conduct a higher-level leadership review, where they go over summaries from these group-level meetings.

Weekly Schedule

- Wednesday: each Epic DRI updates the status section of their epics with the progress. It is important to surface:
  - risks and blockers impacting the project
  - projects that are completed, including a closing summary highlight. These epics will be closed during the Grand Reviews
- Thursday: Group Level Reviews conducted and added as threads in Infrastructure Platforms Top Level epic (see example)
  - Data Access: run by the Data Access Acting Sr. EM and the Group PM
  - Tenant Scale: run by the Tenant Scale Sr. EM and Group PM
  - Production Engineering: run by the Production Engineering Sr. EM and Group PM
  - Software Delivery: run by the Software Delivery Acting Sr. EM and Group PM
  - Developer Experience: run by the Developer Experience Director and a rotation of Product Managers
  - Dedicated: run by the Dedicated Sr. EM and a rotation of of Product Managers
- Friday: Leadership Review, run by the Sr. Director of Infrastructure Platforms and the Director of Product Infrastructure Platforms. Review the group-level summaries added as threads in the Infrastructure Platforms Top Level epic, then conduct a deep dive into one specific group to ensure comprehensive project coverage.
- Friday: Group level and the leadership level reviews are released together in infrastructureplatforms

The review is private streamed to the GitLab Unfiltered channel because the review covers confidential issues. All recordings are made available in the Platforms Grand Review YouTube Playlist

Infrastructure Platforms Leads Demo

The Infrastructure Platforms Leads Demo is an opportunity for sync discussions between Staff+ IC across the Infrastructure Platforms Group to highlight current ongoing efforts underway in the teams they support.
All team members are welcome to join the call, but the emphasis is on Staff+ ICs to present and discuss the work they're focused on, the problems they're experiencing, and solutions they're considering.

The call is recorded to the Infrastructure Platforms Leads Demo Unfiltered Playlist. The agenda can be found in Google Docs.

While the intention is for the call to be made public on GitLab Unfiltered, the default is for it to be published as private.
At the end of the call, a quick vote is held between the attendees and if all agree that the content is SAFE, it can be published as public.

Requests for Help

On the landing page for getting assistance, we ask team-members who need assistance to raise Requests for Help using standard templates.

These issues are raised in the request for help issue tracker and are automatically assigned to the Engineering Manager of the relevant SaaS Platforms team.

The Engineering Manager is expected to:

1. Confirm that the question is not a duplicate and that the answer to the question is not already discoverable in the handbook or the tracker itself.
2. Confirm the urgency of the request.
3. Respond to the help request or assign to an engineer to help with the request.

Slack to GitLab Issue Tracker Integration

In an effort to enhance the tracking and resolution of requests directed to the Infrastructure team, we are evaluating a bot that converts Slack messages in infrastructurelounge channel into GitLab issues.

Workflow Overview

- Acknowledgement: An agent responds with the acknowledgedemoji (· in our case) to acknowledge a Slack message in the Infrastructure Lounge channel.
- Issue Creation: The Slack bot then creates an issue with the acknowledging agent assigned to it.
- Thread Attachment: The Slack thread corresponding to the message is also posted on the created GitLab issue.
- Label Assignment: Agents can further categorize issues by adding label emojis (ops, foundations, observability) in the Slack message. This action automatically assigns the issue to the respective team: Ops, Foundations, Observability.
- Project Tracking: These converted issues are tracked under a dedicated project hosted at Infrastructure Lounge Slack Issue Tracker.
- Issue Closure: Agents/Requester can close the issue when resolved by adding any of the resolvedemojis (green-circle-check,whitecheckmarkor checkedin our case)

Configuration

Agents responsible for handling these issues are defined in a JSON file, which serves as a CI/CD variable. Currently, this file contains a static list of all members of the infrastructure department.

Project and Backlog Management

We use epics and issues to manage our work. Our project management process is shared between all teams in SaaS Plaforms.

Tools

The Platforms section builds and maintains various tools to help deploy, operate and monitor our SaaS platforms. You can view a list of these tools in the Platforms Tools Index.

OKR

We use objective and key results to set goals in alignment with OKRs at GitLab.
Our OKR process is shared between all teams in Saas Platforms.

Hiring and Interviewing

Our hiring process is shared between all teams in Infrastructure Plaforms.

This Infrastructure Platforms Interviewing Guide offers more detail on some of our regular openings, interview process and other useful information related to applying to jobs with us. More information on our current openings can be found on the careers page.

Platforms Learning Path

All team members are encouraged to schedule time for personal development. The following links may help you get started with Platforms-relevant learning. Please add your own contributions to this list to help others with their personal development.

Learn about Infrastructure Platforms, and its groups

Group	Topic
SaaS Platforms	Product direction
Delivery Group	Delivery Group
Production Engineering Group	Production Engineering
Dedicated Group	Dedicated Group
Tenant Scale	Group Page

Learn about tools and technologies used within Platforms

1. Jsonnet tutorial

Common Links

- How we do Incident Management for GitLab.com
- GitLab.com status information

Other Slack Channels

- production
- infrastructure-lounge
- incidents
- announcements
- feedalerts-general

General Issue Trackers

- Production Engineering issue queue
- Production incidents, and changes
- Delivery
- Observability
- Tenant Scale
- Runway

Resources

- Production Architecture
- Operational Runbooks
- Environments
- Monitoring
- Readiness Reviews
- Infrastructure Platforms Standards
- Career development and Internships in Infrastructure Platforms

Other Pages

- On-call Handover
- SRE Onboarding
- GitLab.com data breach notification policy
- Coding at scale

SECTION: engineering/infrastructure-platforms/alert-playbook-management.md

Purpose

During an incident, playbooks are vital to the engineer on call (EOC) in resolving an alert. Having all of the salient information laid out in one place saves the EOC time in diagnosing and resolving the incident. It empowers the EOC with a set of standard steps for responding to incidents. Additionally, it can greatly reduce the stress of dealing with an alert when working on an unfamiliar service.

Our current runbooks tend to be more architectural documents or contain many links to unrelated information without much troubleshooting guidance. Moreover, they are primarily service-based runbooks. This means the runbooks are not necessarily useful in determining what an alert means or how to handle it. Our transition to alert-based playbooks will improve initial response times as the information relevant to the alert as well as other contextual information will be immediately available.

Expectations

A playboook should use the playbook template and follow the guidence within. The playbook should include the following sections:

1. An overview section including what the alert means and what action the EOC is expected to take.
2. Information on the service and who owns the service.
3. Information on the metrics that cause the alert to fire and an explanation on why these metrics were chosen.
4. A description of the alert behavior such as expected frequency and guidelines on silencing the alert.
5. Guidance on the likely severity of the incident and tips on how to determine severity.
6. Verification information to ensure the alert is accurate such as links to dashboards and log

queries relevant to the alert.
7. Links to recent changes that may have caused this alert. This could be links to change management issues with the proper tags, recent MRs in the chef-repo or gitlab-helmfiles repo. If relevant, also include information on rolling back said changes.
8. Information on the troubleshooting steps to identify the issue. This is one of the most important sections and should include as much details as possible. This could include relevant servers to log in to, commands to run, dashboards to check, useful scripts, or any other advice in troubleshooting.
9. Possible resolutions to the alert, often links to previous resolved incident issues involving this alert.
10. Any external or internal dependencies that could cause this alert. For example a database problem might cause a sidekiq alert.
11. Escalation steps and procedures such as when and where to escalate.
12. Definitions including linking to the source of the alert and any guidance on when and how to tune the alert.

Guidelines on Playbook Management and Creation

- Create playbooks in the relevent service directory under /alerts/ such as docs/<service name>/alerts/ in the Runbooks Repo using the playbook template.
- Condense alerts into a single playbook when the difference between alert names is only a suffix with a header line indicating that the playbook covers multiple alerts.
  - For example, WALGBaseBackupDelayed and WALGBaseBackupFailed are combined into one playbook named WALGBaseBackup. The majority of the details will be the same, but the difference between "Delayed" and "Failed" need to be discussed.
- Update the links that are attached to the alerts. Alerts are defined in either the mimir-rules-jsonnet or mimir-rules directories in the Runbooks repo.
- When adding a new alert, the service owners adding the alert should create a playbook before the alert is added.
- After modifying an alert or playbook, you'll need to run make generate in the runbooks repo. Instructions on how to prepare the runbooks repo can be found in the contributor onboarding in the README.
- When creating the merge request, use the template alert-playbook-template for the text of the merge request.
- Reach out to the Production Engineering - Ops team in ginfraops Slack channel if you need assistance.

Important Links

- Playbook Template
- MR template for new playbooks

SECTION: engineering/infrastructure-platforms/careers.md

Engineers within the Infrastructure Platforms department

Career growth within Infrastructure Platforms

Infrastructure Platforms follows the Engineering department approach to career growth with department-specific exectations for each job level

It's important for you to take an active role in shaping your career path. Consider having open conversations with your manager about your professional goals and aspirations. These discussions can help ensure you're both aligned on your development priorities and can work together to create opportunities that support your growth.

Talent Assessment

The annual company Talent Assessment event provides another opportunity to gain insight on progress towards both near-term and longer, career goals.

Interning with Infrastructure Platforms

Cross reference for Internship for Learning

There is a reasonable amount of interest to intern with Infrastructure so we wanted to put a brief landing section in the handbook. If you are interested in interning with Infrastructure Platforms, we would ask that you talk to your current manager and create an issue per the mentioned process. This can help us track who is helping on what and what they would like to learn. One note, in some cases, you may need to submit access-requests to get access to certain systems to help with work.

Don't know what to learn? Check out the SRE I and the SRE II Training Modules, these cover a range of topics related to how we do Reliability Engineering at GitLab. It is recommended that you come to your internship prepared, and by completing both modules, you'll ensure a richer internship experience as you'll make the best use of your time during your internship with an extra focus on more advanced topics.

Additionally, we have a broad set of technical skills listed in the job description for SRE, if you are looking for ideas on what to learn, set up a coffee chat with a team member or one of the managers.

If you're interested in learning more about SREs in general, you may wish to read Alice Goldfuss's blog post, "How to Get Into SRE." This blog post is an overview of all sorts of SRE related things, some of which may not be in use here at GitLab. However, it can be a valuable tool to learn more about SREs in general, and it links to many resources that you can use to learn more about a variety of SRE related topics.

Specific ways to work in the internship:

1. Pair up with an engineer on a particular issue or for our project work.
2. Shadow the On-Call in your similar timezone for 2 weeks
3. Shadow the Delivery team to learn more about how we release to GitLab.com

---
SECTION: engineering/infrastructure-platforms/change-management.md

Purpose

Change Management has traditionally referred to the processes, procedures, tools and techniques applied in IT environments to carefully manage changes in an operational environment: change tickets and plans, approvals, change review meetings, scheduling, and other red tape.

In our context, Change Management refers to the guidelines we apply to manage changes in the operational environment with the aim of doing so (in order of highest to lowest priority) safely, effectively and efficiently. In some cases, this will require the use of elements from traditional change management; in most cases, we aim to build automation that removes those traditional aspects of change management to increase our speed in a safe manner.

Our overriding objective is maximize changes that avoid traditional aspects of change management, which is an iterative process that will evolve over time. Success is measured by our ability to safely execute changes at the speed required by our business needs.

Scope

Changes are defined as modifications to the operational environment, including configuration changes, adding or removing components or services to the environment and cloud infrastructure changes. Our Staging environment is crucial to our GitLab.com release process. Therefore, Staging should be considered within scope for Change Management, as part of GitLab's operational environment. Application deployments, while technically being changes, are excluded from the change management process, as are most, but not all, feature flag toggles.

Changes that need to be performed during the resolution of an Incident fall under Incident Management.

Roles and Responsibilities

Role	Responsibility
GitLab Team Members	Responsible for following the requirements in this procedure
Infrastructure Team	Responsible for implementing and executing this procedures
Infrastructure Management (Code Owners)	Responsible for approving significant changes and exceptions to this procedure

Does my change need a change request?

Does your change:

- Carry some risk?
  > To help answer this question: has this change been attempted before? how extensively has this change been tested in non-prod environments? how confident are you in the evidence that this will not cause an issue in production?
- Require deployments and/or feature flags to be paused?
- Require manual steps?
- Affect multiple departments and need increased visibility?

If you answered YES (or even "maybe") to any of the questions above, then you should follow the

change request workflow. The intent of the workflow is to reduce the likelihood of a change making it to production that has a negative impact by increasing visibility, and in cases where a change of this nature does make it to production, having a reviewed rollback plan ready to be executed means we can get back to a good state as quickly as possible.

Examples of changes that should follow the change request workflow:

- Implement site wide rate limit in Cloudflare.
- CSP changes in production.
- Drop an index from a table in staging

When you don't need a change request

Non-exhaustive list of the types of changes that do not need a change request:

- Merging a merge request that will automatically be deployed and the rollback is reverting the merge request.
- A merge request that is considered low risk.
- Minor dependency updates that aren't critical to our environment.
- Feature toggles that are done via configuration and are not considered risky.

Examples:

- Add service dependency on alerting
- Small cookbook bumps
  - Use your judgement: it really depends on the change made to the cookbook to determine if the cookbook version bump should follow our change management workflow or not.

When you are not sure

- Ask for opinions in infrastructure-lounge or sproductionengineering
- Open a change management issue, err on the side of caution.

Change Request Workflows

Plan issues are opened in the production project tracker via the change management issue template. Each issue should be opened using an issue template for the corresponding level of criticality: C1, C2, C3, or C4. It must provide a detailed description of the proposed change and include all the relevant information in the template. Every plan issue is initially labeled ~"change::unscheduled" until it can be reviewed and scheduled with a Due Date. After the plan is approved and scheduled it should be labeled ~"change::scheduled" for visibility.

To open the change management issue from Slack issue the following slash command:

text
/change declare

Creating the Change issue from Slack will automatically fill in some fields in the description.

Aborting a Change

In the case a change is rolled back or if it will not be completed apply the change::aborted label and close the issue.
A new change should be declared if the change is retried on a later date.

Change Criticalities

C1 and C2 change requests will automatically have the labels, ~blocks deploys and ~blocks feature-flags applied. When these critical change requests are labeled change::in-progress, they will block deploys, feature flag changes, and potentially other operations. Take particular care to have good time estimates for such operations, and ideally have points/controls where they can be safely stopped if they unexpectedly run unacceptably long. Remove the ~blocks deploys and/or ~blocks feature-flags if the proposed change will not be negatively impacted by said operations.

In particular for long running Rails console tasks, it may be acceptable to initiate them as a C2 for approvals/awareness and then downgrade to a C3 while running. However consider carefully the implications of long running code over multiple deployments and the risks of mismatched code/data storage over time; such a label downgrade should ideally have at least 2 sets of eyes (SREs/devs) assess the code being exercised for safety, and management approval is recommended for visibility.

Criticality 1

These are changes with high impact or high risk. If a change is going to cause downtime to the Production environment, it is always categorized a C1.

Examples of Criticality 1:

1. Any changes to Postgres hosts that affect DB functionality (e.g. quantity of nodes, changes to backup strategy, changes to replication strategy, configuration changes, etc.).
1. Major architectural changes to Infra as code (IaC).
1. Major IaC changes to pets - Postgres, Redis, and other Single Points of Failure.
1. Changes of major vendor or service provider (e.g. CDN, transactional mail provider, DNS, etc.).
1. Major version upgrades (major definition following www.semver.org) of tooling (e.g HAProxy, AlertManager, Chef, etc.).